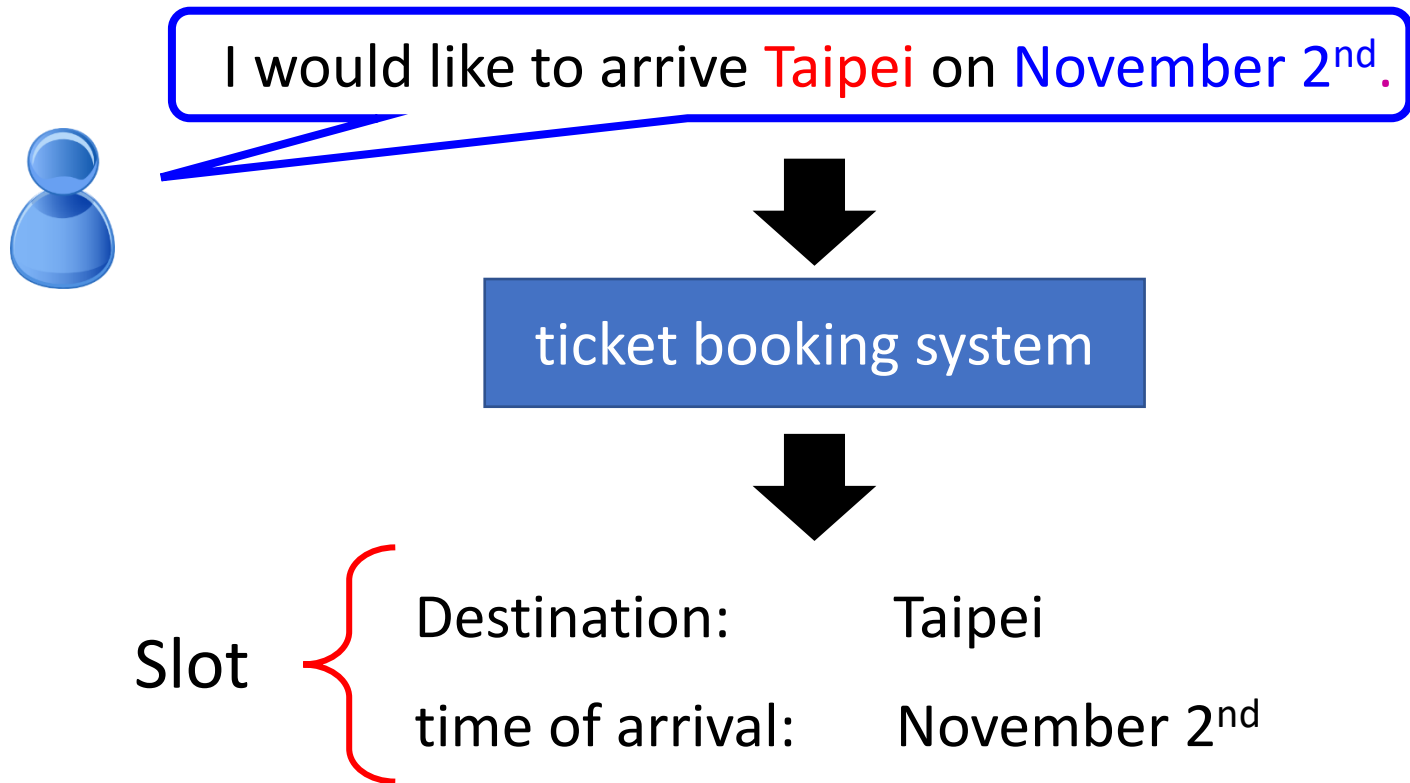


Recurrent Neural Network (RNN)

Example Application

- Slot Filling

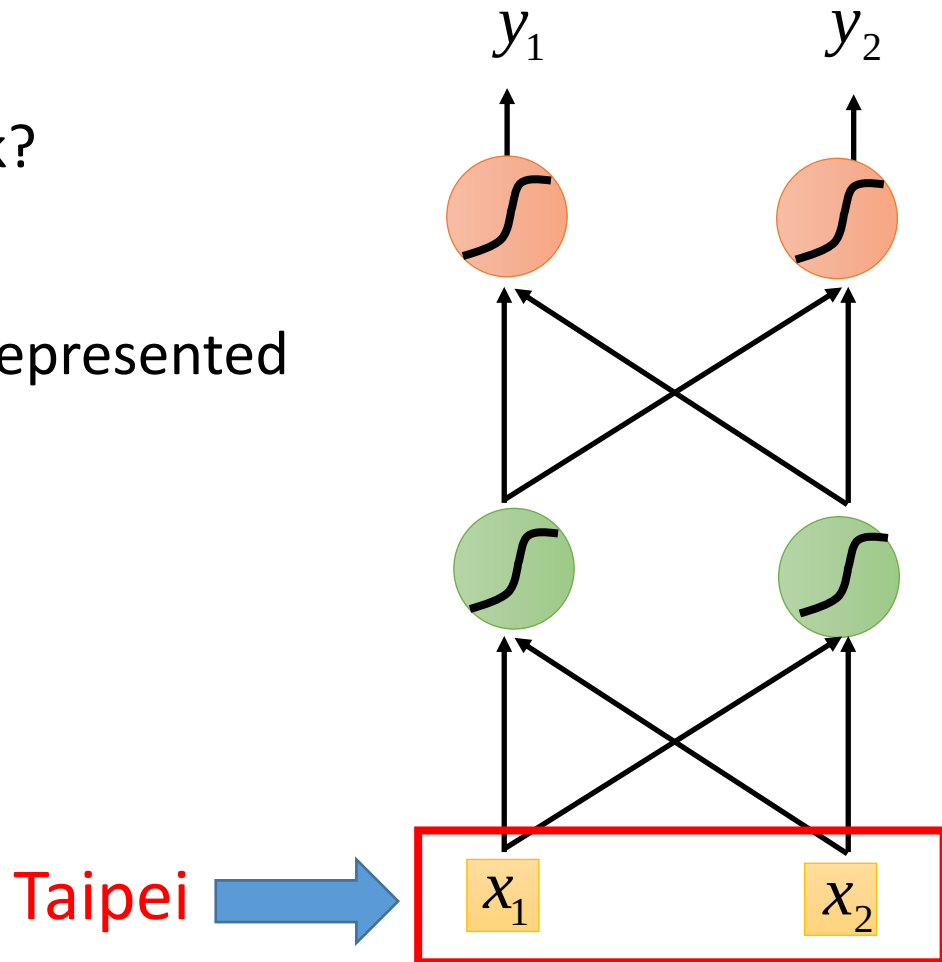


Example Application

Solving slot filling by
Feedforward network?

Input: a word

(Each word is represented
as a vector)



1-of-N encoding

How to represent each word as a vector?

1-of-N Encoding lexicon = {apple, bag, cat, dog, elephant}

The vector is lexicon size.

Each dimension corresponds
to a word in the lexicon

The dimension for the word
is 1, and others are 0

apple = [1 0 0 0 0]

bag = [0 1 0 0 0]

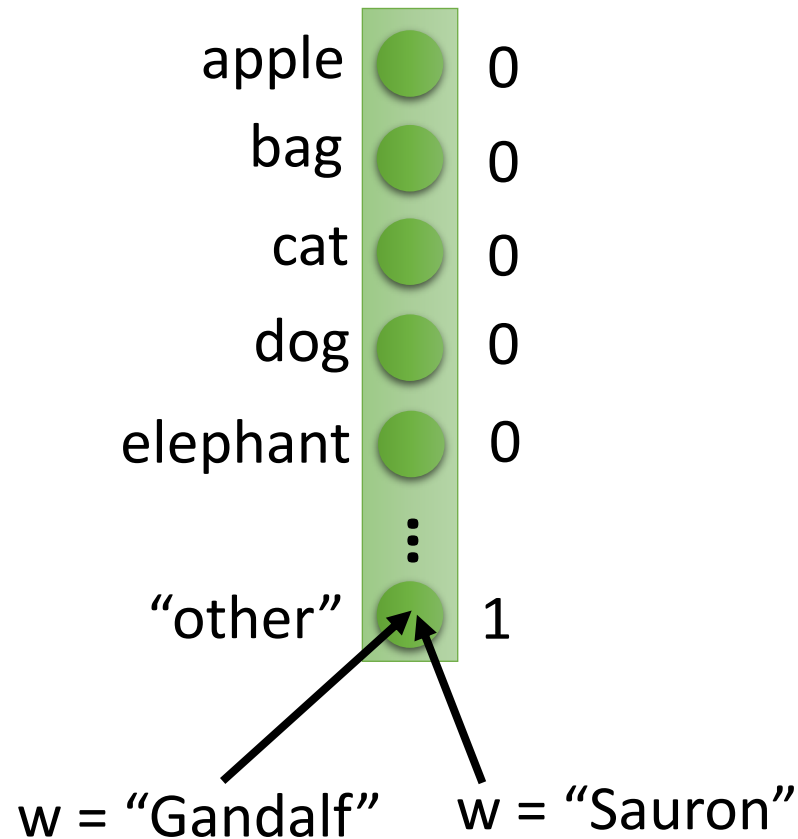
cat = [0 0 1 0 0]

dog = [0 0 0 1 0]

elephant = [0 0 0 0 1]

Beyond 1-of-N encoding

Dimension for “Other”



Example Application

Solving slot filling by
Feedforward network?

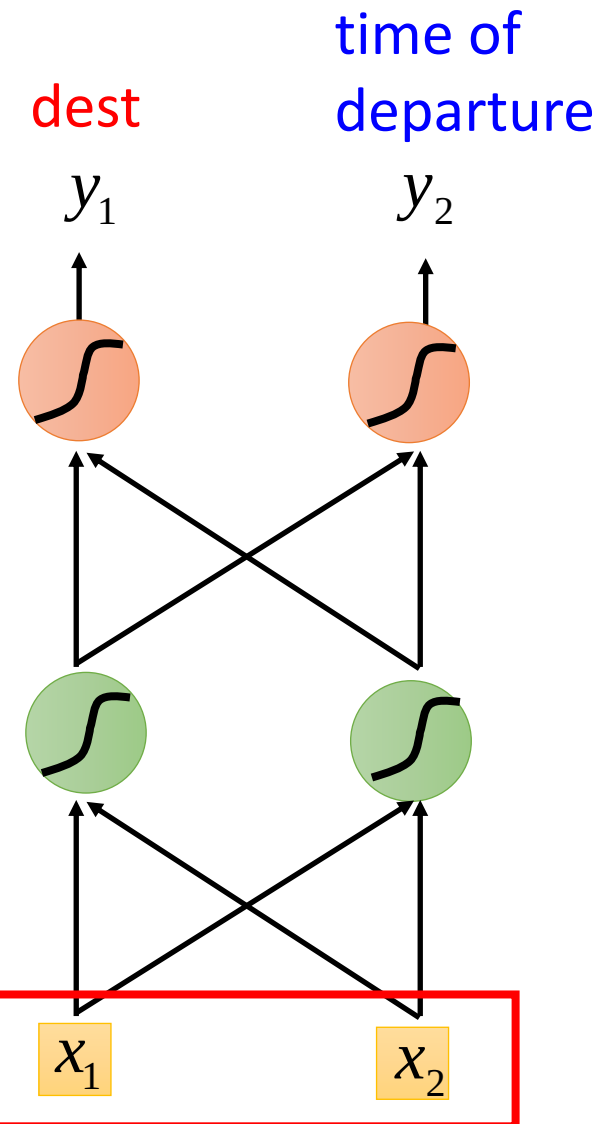
Input: a word

(Each word is represented
as a vector)

Output:

Probability distribution that
the input word belonging to
the slots

Taipei →



Example Application

arrive Taipei on November 2nd

other

dest

other

time

time

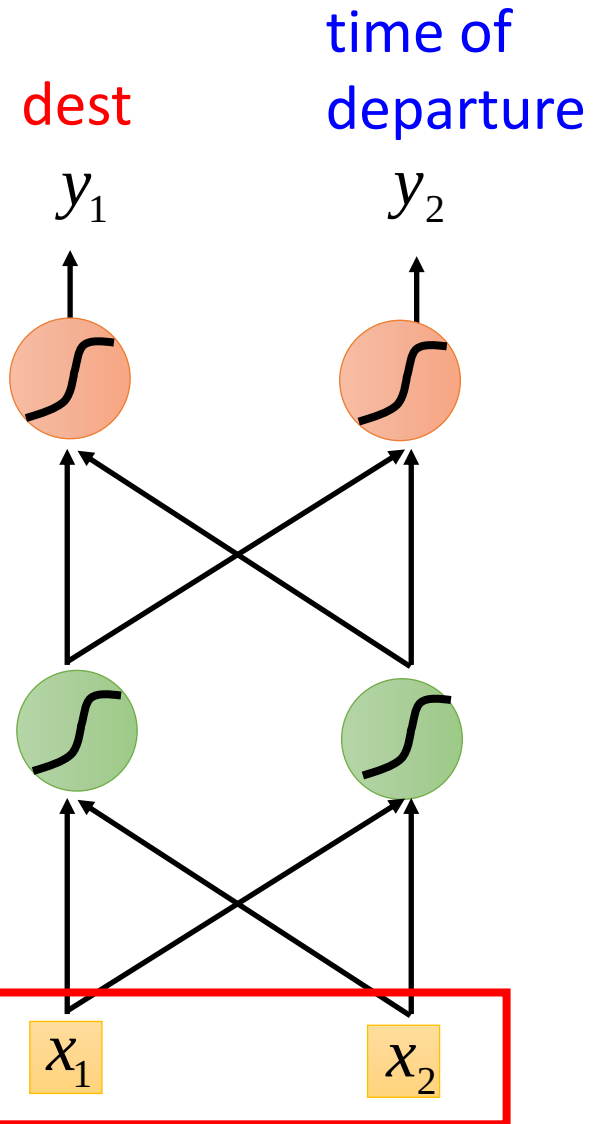
Problem?

leave Taipei on November 2nd

place of departure

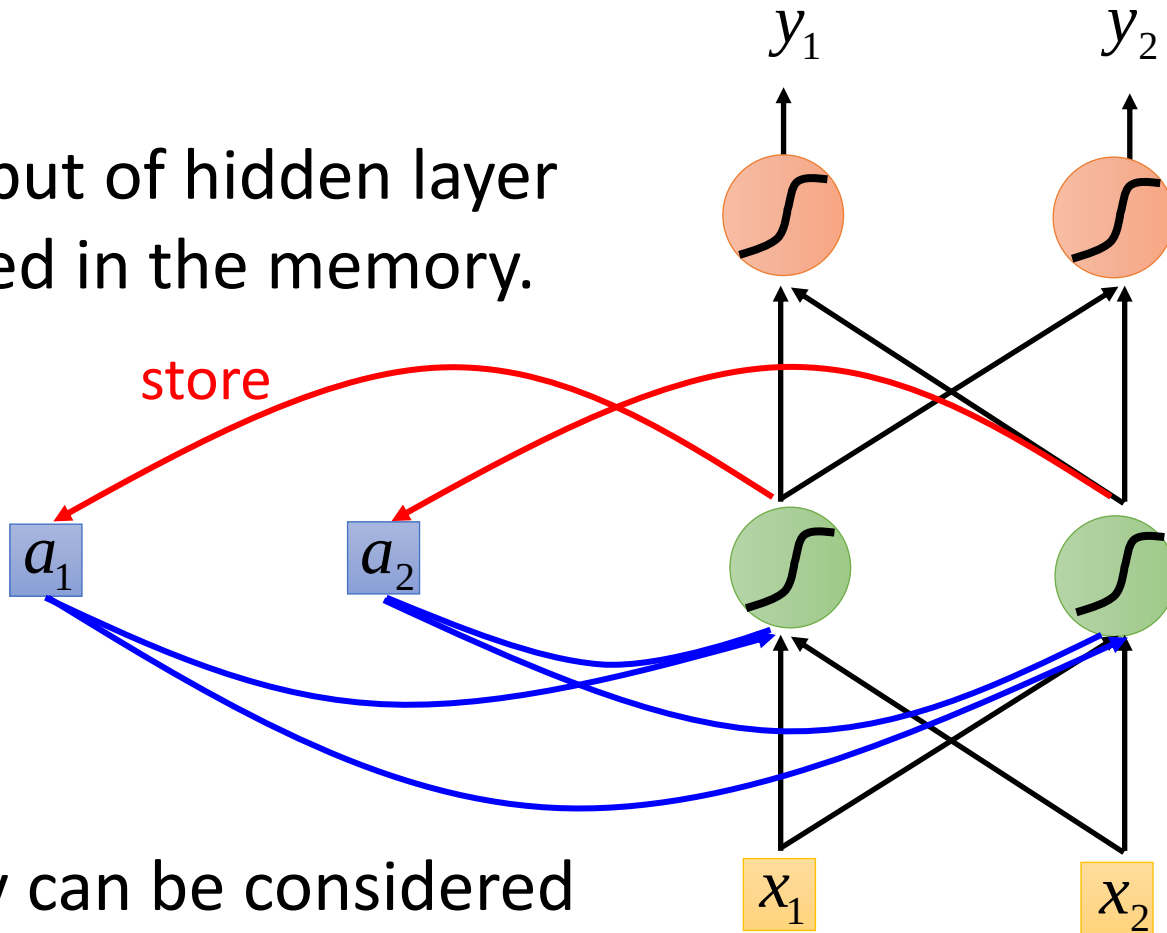
Neural network
needs memory!

Taipei



Recurrent Neural Network (RNN)

The output of hidden layer are stored in the memory.

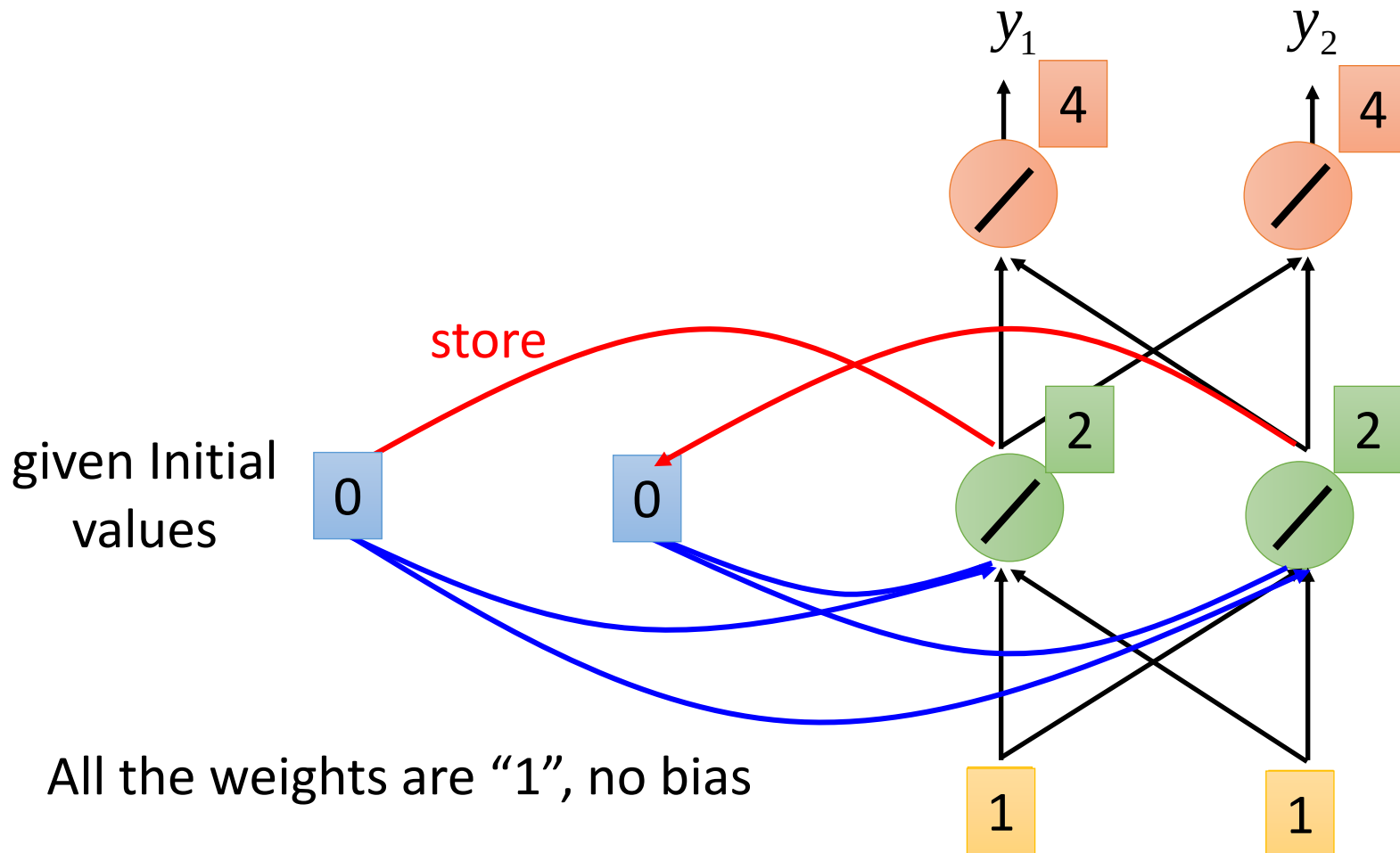


Memory can be considered as another input.

Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$



given Initial
values

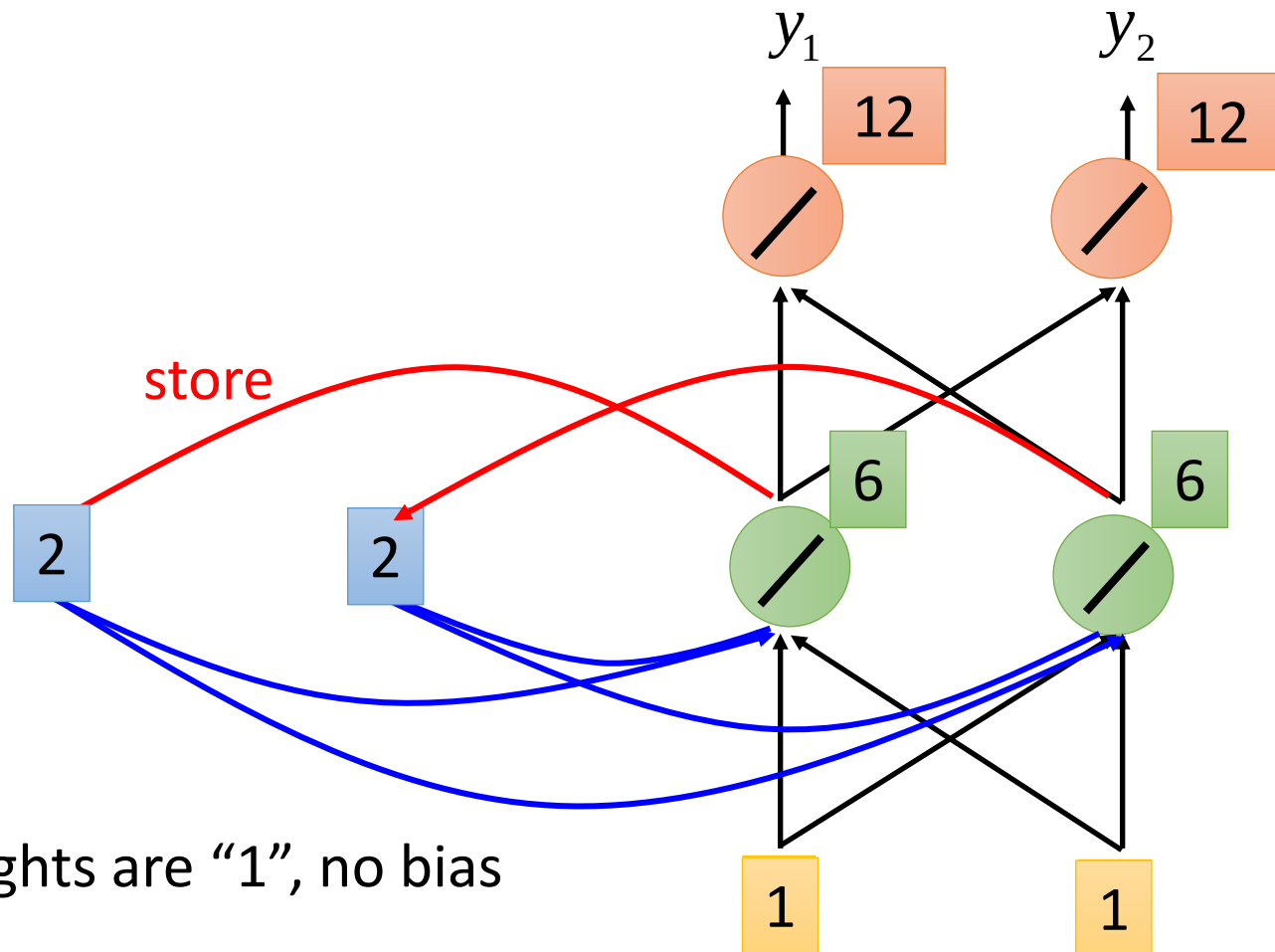
All the weights are "1", no bias

All activation functions are linear

Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$ $\begin{bmatrix} 12 \\ 12 \end{bmatrix}$



All the weights are “1”, no bias

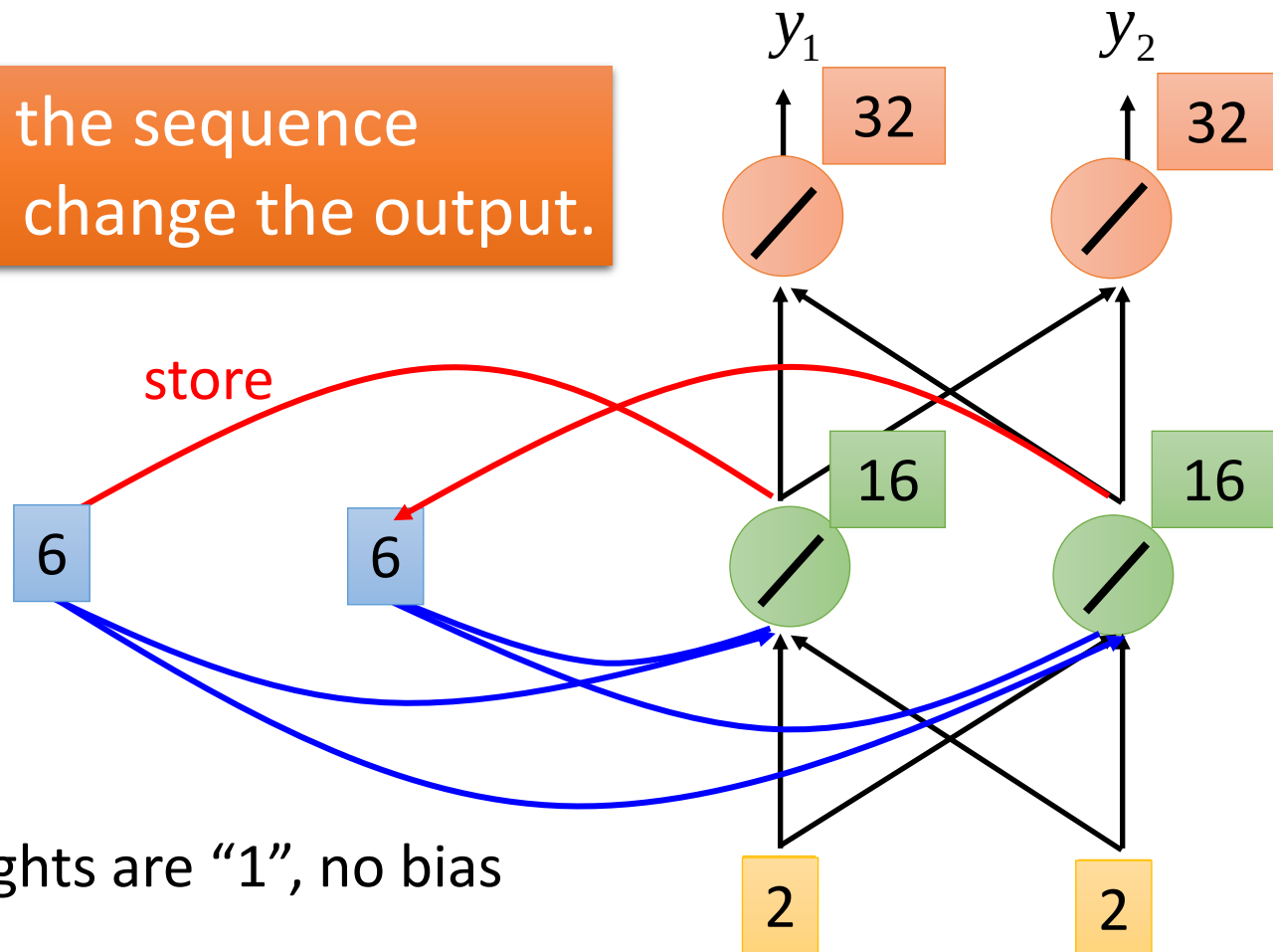
All activation functions are linear

Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$

output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$ $\begin{bmatrix} 12 \\ 12 \end{bmatrix}$ $\begin{bmatrix} 32 \\ 32 \end{bmatrix}$

Changing the sequence order will change the output.



All the weights are "1", no bias

All activation functions are linear

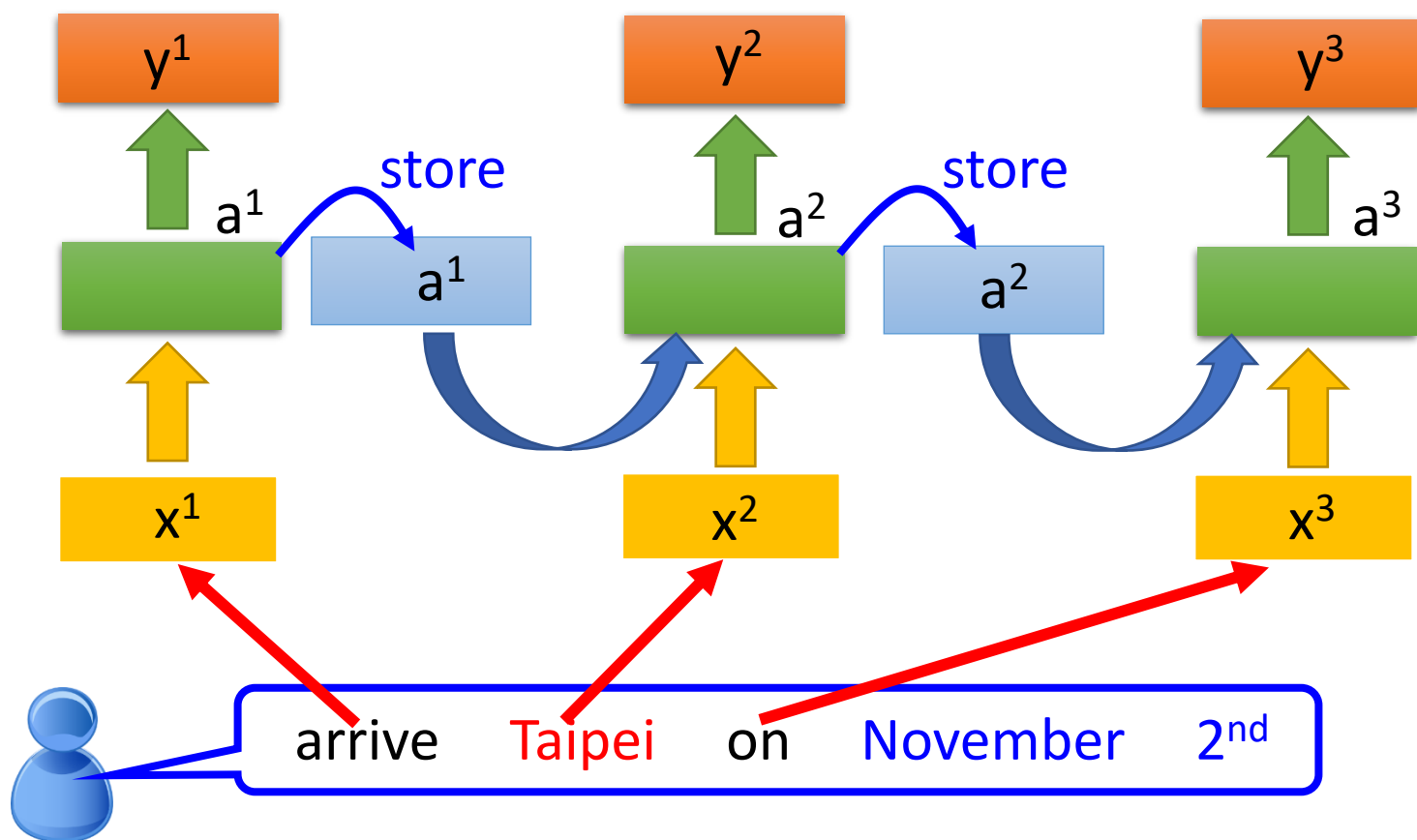
RNN

The same network is used again and again.

Probability of
“arrive” in each slot

Probability of
“**Taipei**” in each slot

Probability of
“on” in each slot



RNN

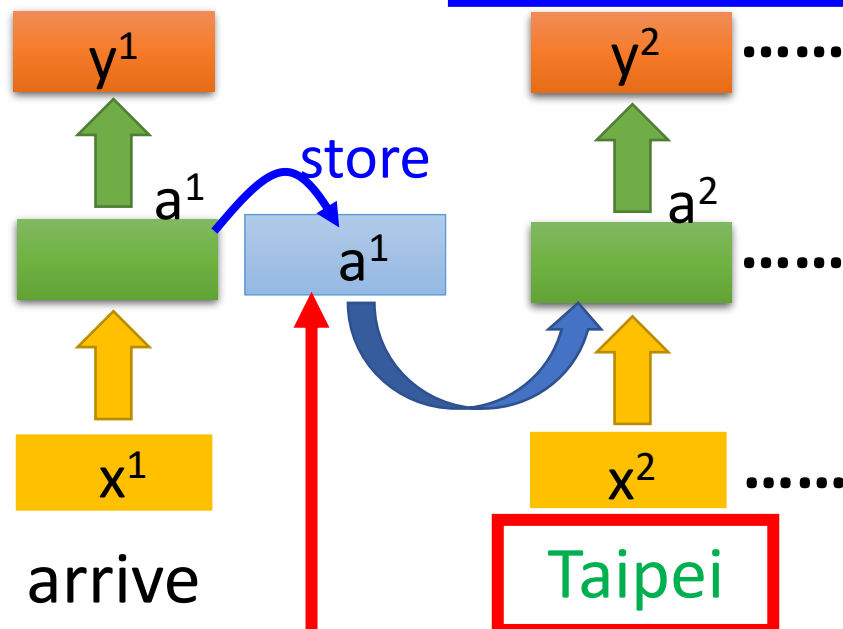
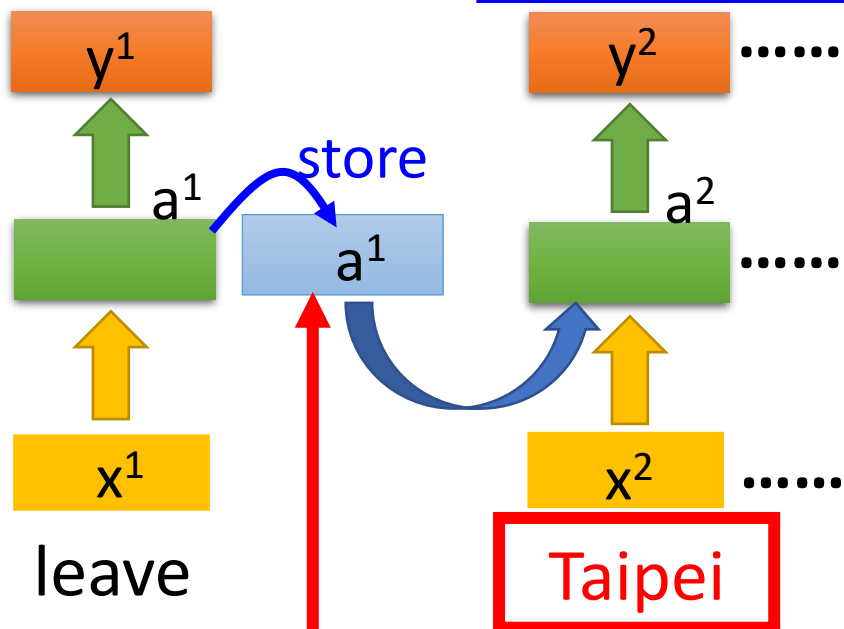
Different

Prob of “leave”
in each slot

Prob of “**Taipei**”
in each slot

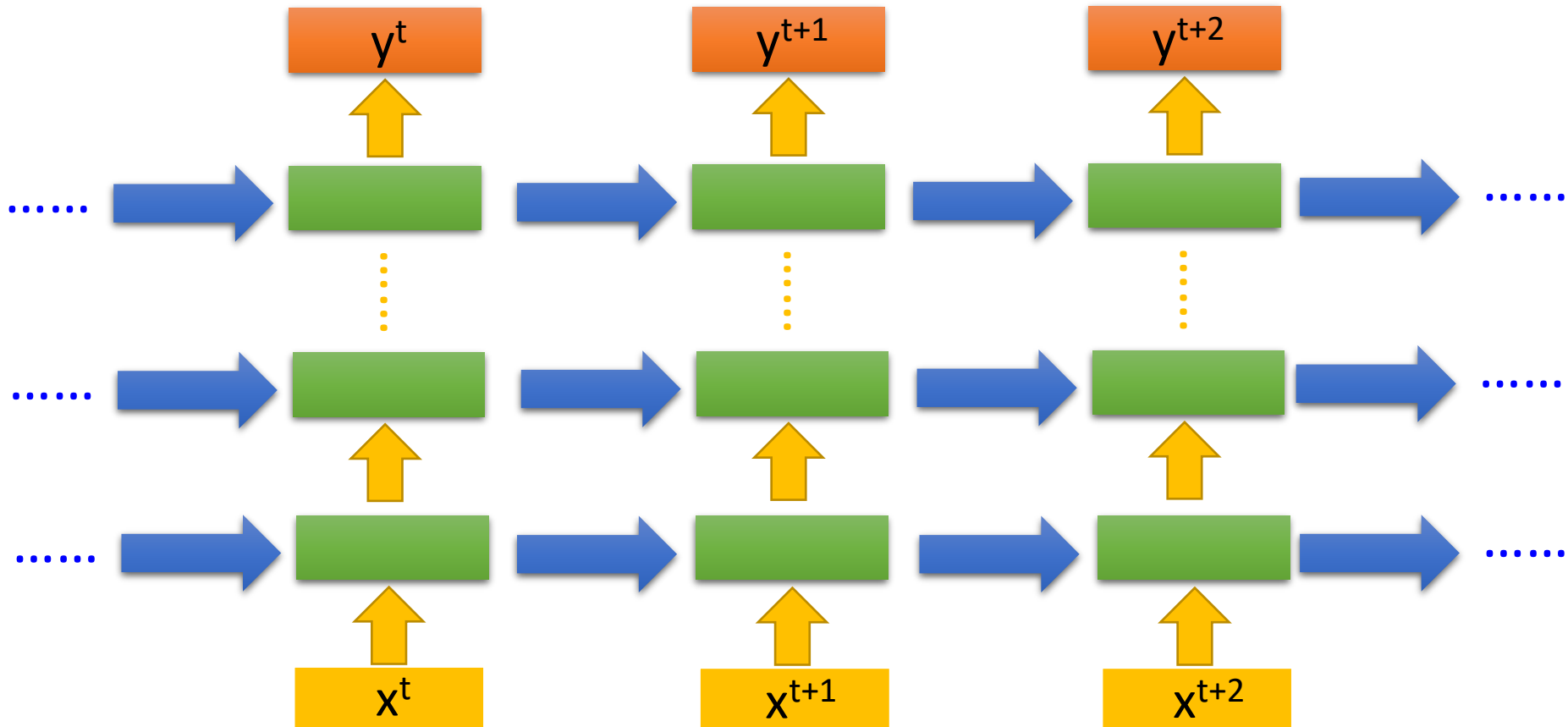
Prob of “arrive”
in each slot

Prob of “**Taipei**”
in each slot



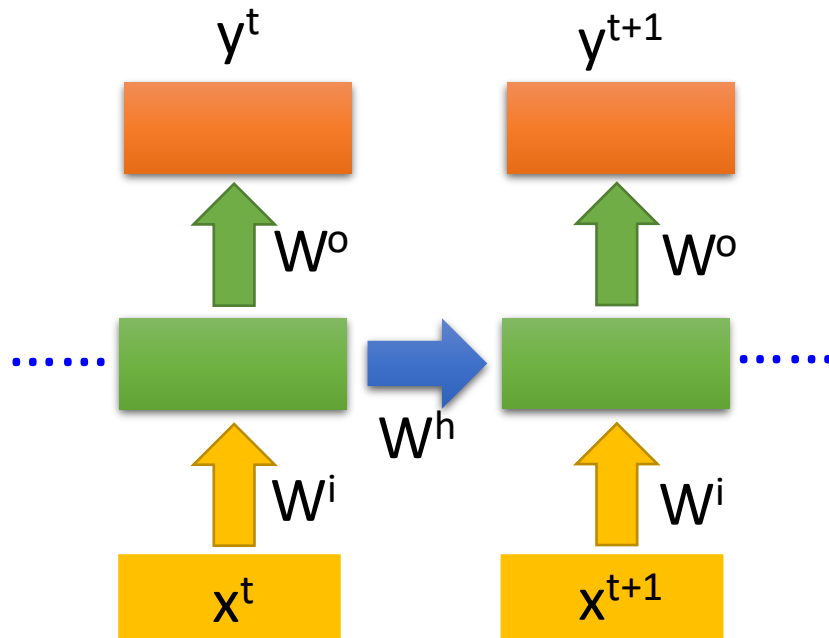
The values stored in the memory is different.

Of course it can be deep ...

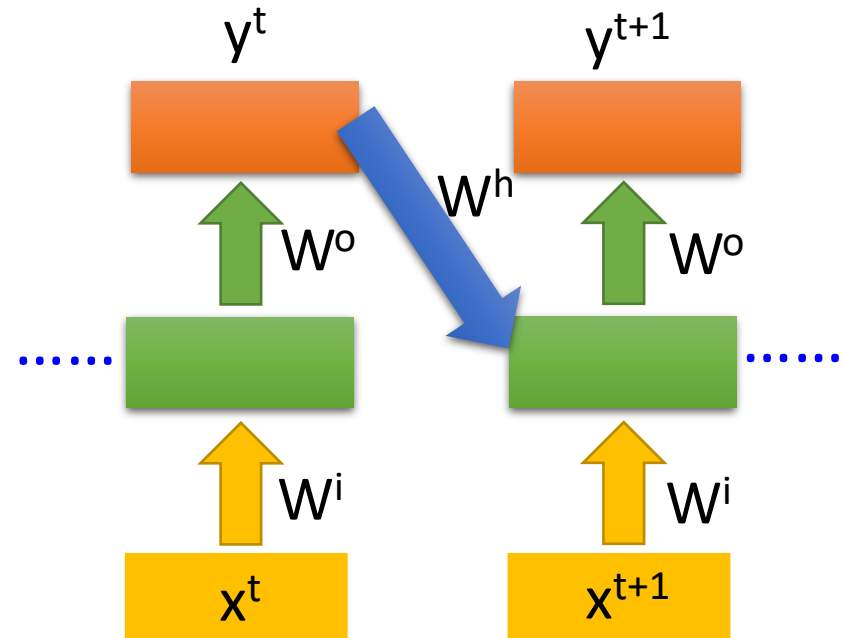


Elman Network & Jordan Network

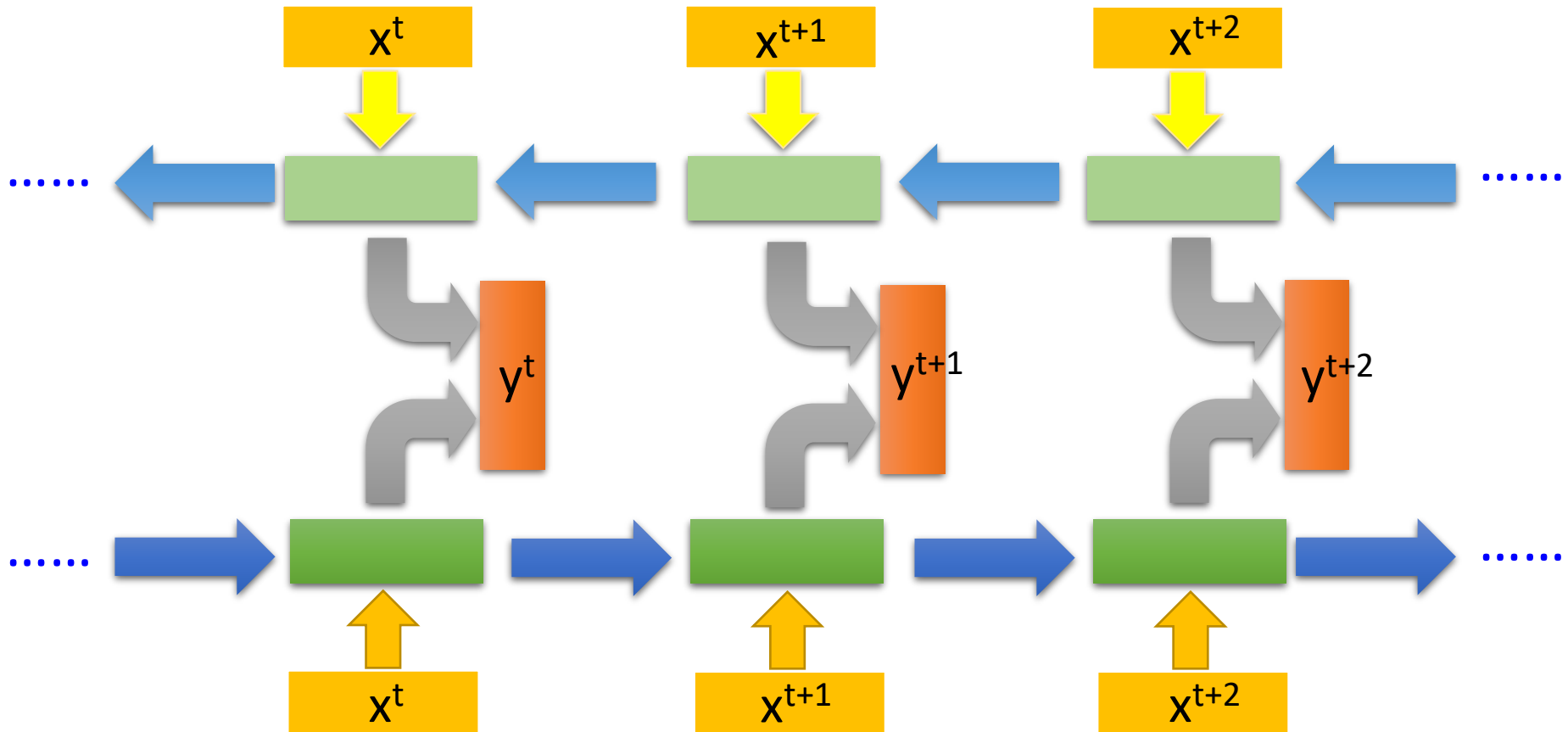
Elman Network



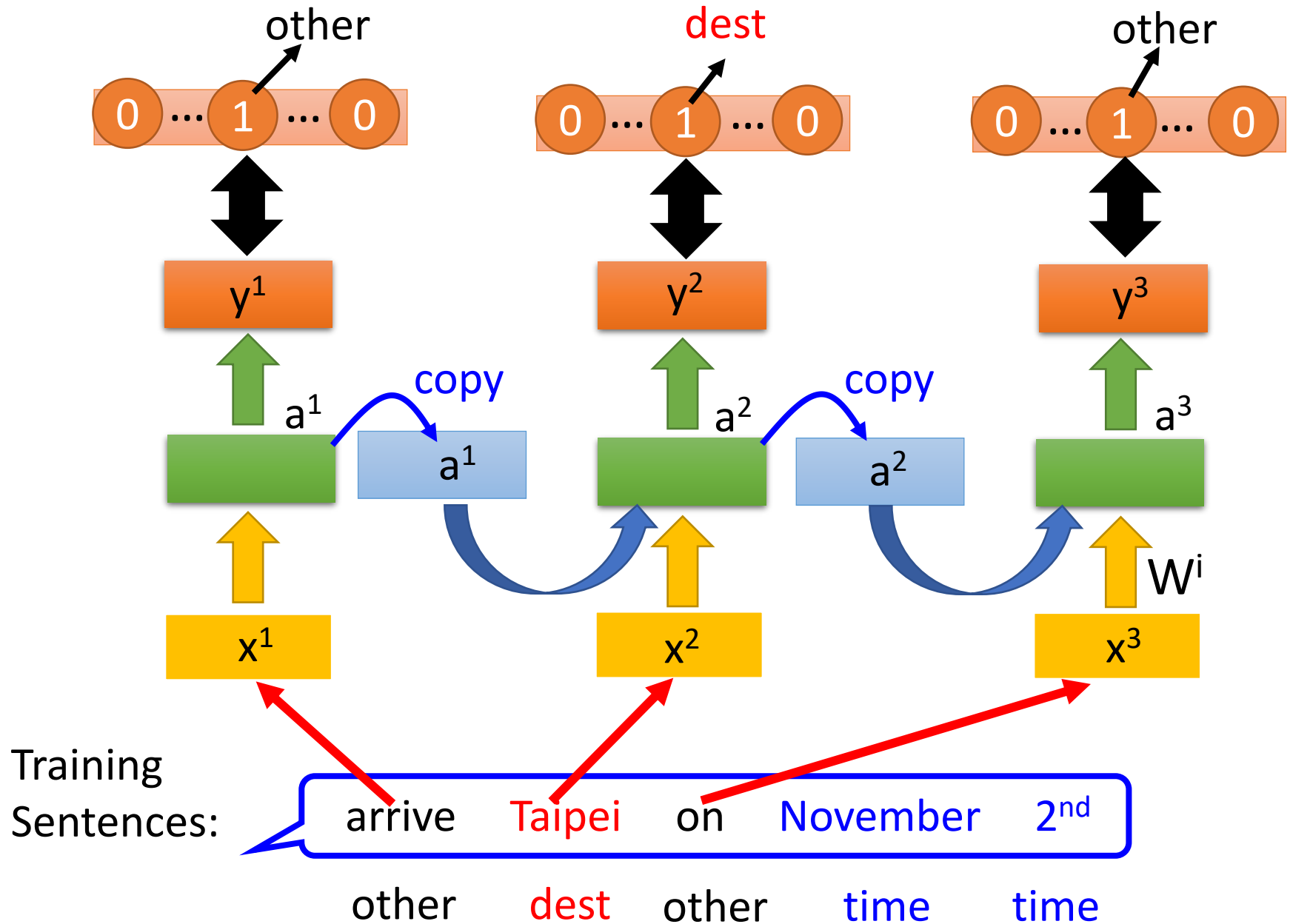
Jordan Network



Bidirectional RNN

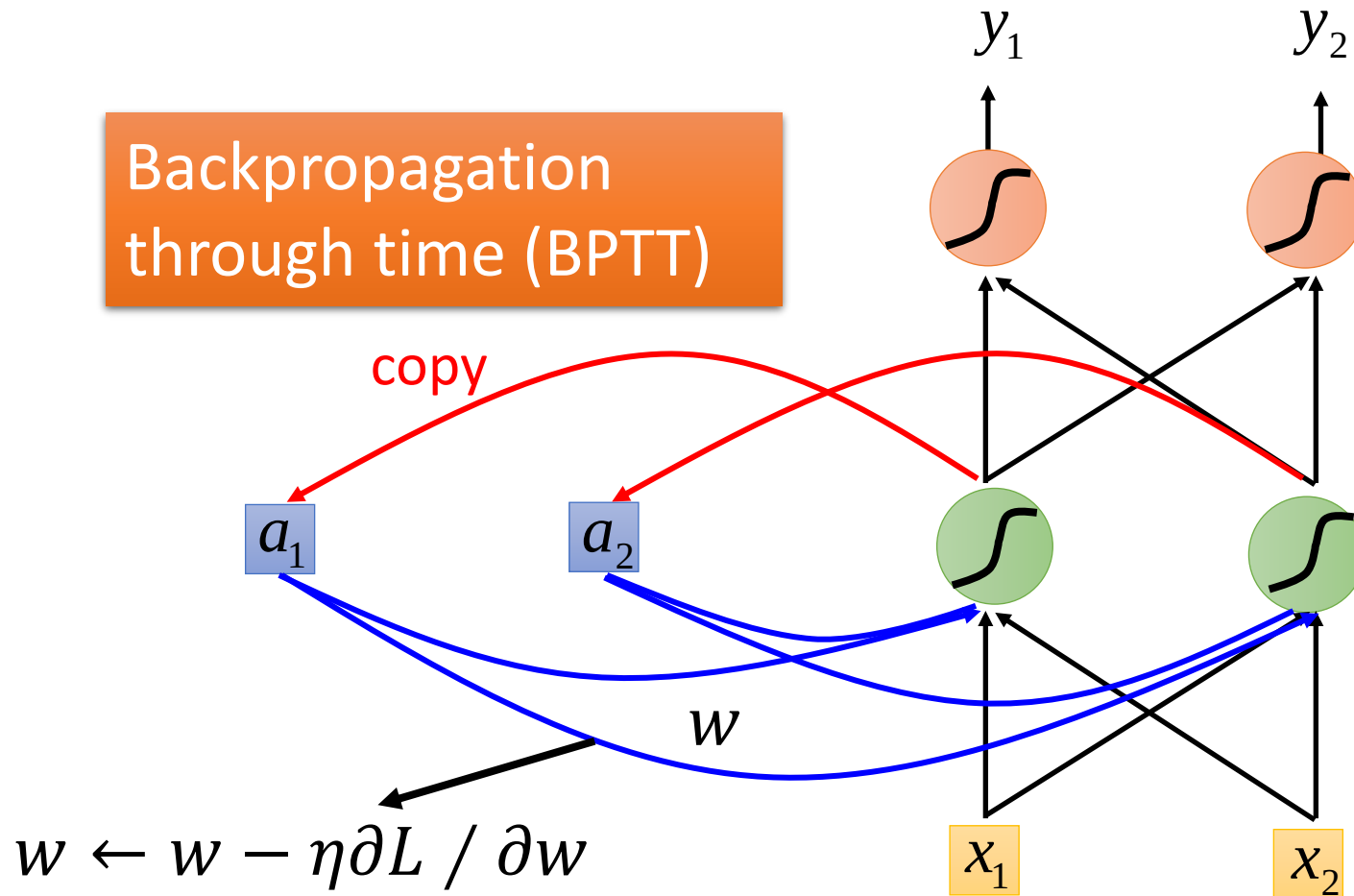


Learning Target



Learning

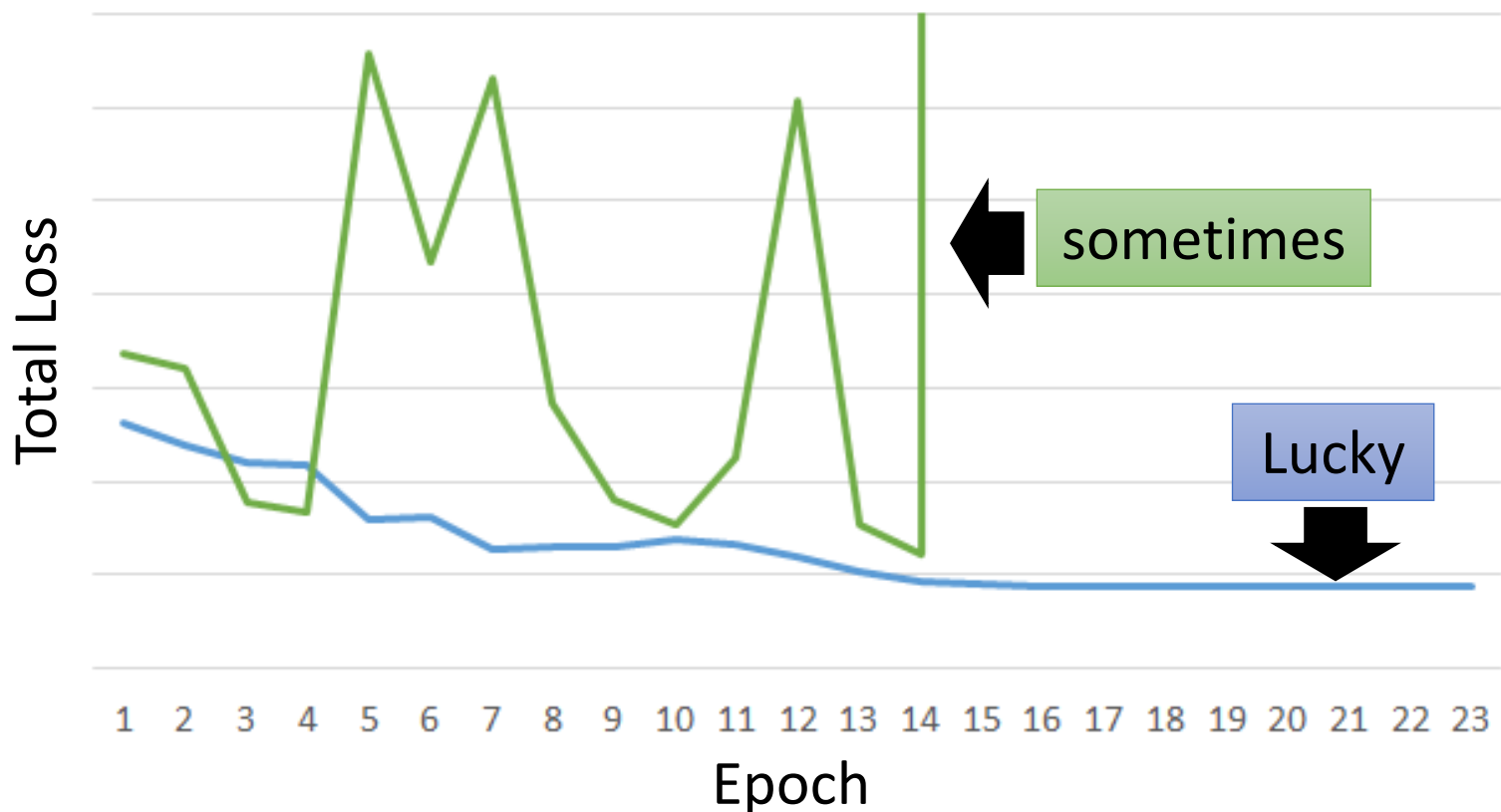
Backpropagation
through time (BPTT)



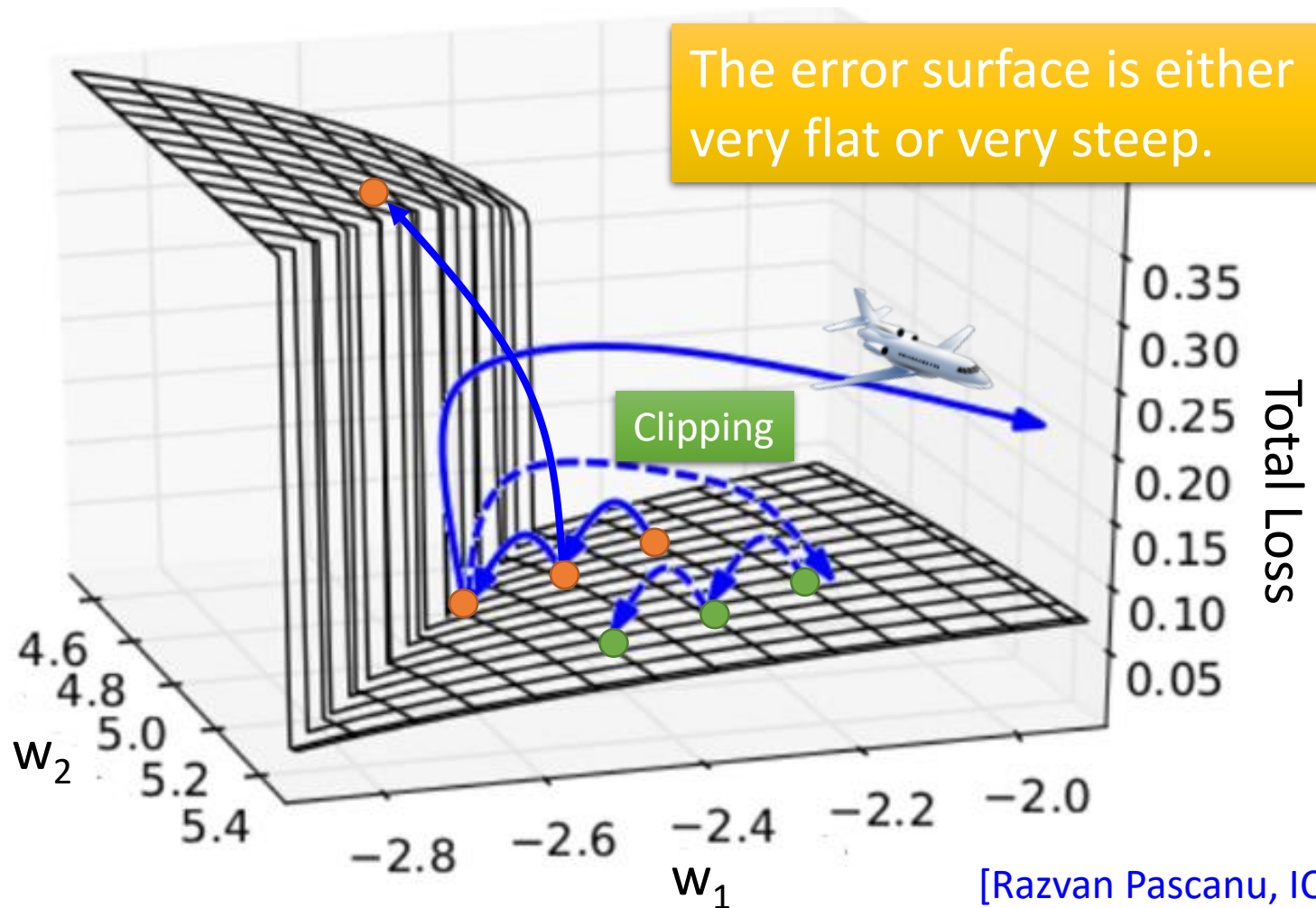
Unfortunately

- RNN-based network is not always easy to learn

Real experiments on Language modeling



The error surface is rough.



Why?

$$\begin{array}{ll} w = 1 & \longrightarrow y^{1000} = 1 \\ w = 1.01 & \longrightarrow y^{1000} \approx 20000 \end{array}$$

$$\begin{array}{ll} w = 0.99 & \longrightarrow y^{1000} \approx 0 \\ w = 0.01 & \longrightarrow y^{1000} \approx 0 \end{array}$$

Large
 $\partial L / \partial w$

Small
Learning rate?

small
 $\partial L / \partial w$

Large
Learning rate?

Toy Example

